

SAGEPILOT: Latency-Aware Orchestration for Multi-Agent LLM Workflows on Heterogeneous GPUs

Abstract

Concurrent multi-agent workflows expose future dependencies and serving-state requirements while running on heterogeneous GPU pools with time-varying load, model residency, and resource availability. The logical workflow defines the required computation, whereas its physical scheduling units, model-lifecycle actions, resource ordering, and placement must be selected according to the observed pool state. We present SAGEPILOT, which uses workflow forecasts to construct and optimize a physical execution graph. SAGERADAR predicts device-specific activation latency, peak memory, and model-loading cost, then propagates these estimates through workflow dependencies to forecast activation readiness and future model demand. SAGEGRAPH constructs semantics-preserving fusion and model-lifecycle alternatives, while SAGEWEAVER jointly optimizes their selection, placement, and scheduling based on the live pool state. Across a workload spanning three workflow scenarios on a heterogeneous GPU pool, SAGEPILOT reduces end-to-end makespan and overall p95 completion latency under burst arrivals by up to 36.8% and 25.9%, respectively, over state-of-the-art workflow schedulers. It also saves up to 24.63 GPU-s per completed session.

Keywords: Multi-Agent Workflows, LLM Serving, Graph Optimization, Heterogeneous GPUs, Adaptive Scheduling

1 Introduction

LLMs such as DeepSeek [15], Qwen [41], and LLaMA [36] increasingly power compound AI systems that orchestrate model calls, tools, and control flow [46]. LangGraph [23], AutoGen [40], and AgentScope [11] express these applications as stateful Agent graphs. Software-development systems coordinate planners, coders, testers, and reviewers [16, 30], while information-seeking systems dispatch parallel researchers and integrate their results [4]. Serving these applications therefore means executing concurrent multi-Agent workflows whose dependencies, model state, and resource demands unfold over time.

Agent workflows increasingly process confidential inputs and proprietary data, making private deployment a practical requirement in privacy-sensitive sectors [5, 20]. Cost constraints keep these deployments on small, non-elastic GPU pools; fixed capacity and device heterogeneity make physical execution a first-order decision. Agent roles invoke

different models and request shapes, and the same activation can vary substantially in latency, memory demand, and feasibility across GPUs [38]. A flexible review Agent can occupy the only high-capacity GPU required by a long-context synthesizer; model weights, KV caches, prefixes, and engine state can meanwhile remain useful to later activations. Because model upgrades and incremental GPU procurement also leave sparse model-device measurements, the runtime must decide not only *when* an Agent can run, but *how* and *where* to realize it.

Workflow progress exposes many physical choices before every downstream Agent becomes runnable. Once its branch, role, model, and eligible devices are determined, the runtime can prefetch its deployment or retain and reclaim resident replicas while a predecessor still runs. It can also fuse consecutive same-deployment activations, order independent units to avoid resource conflicts, and coordinate compatible model lifecycles across workflows. These choices preserve the logical workflow but change loading, residency, peak memory, and critical-path time; poor choices instead evict useful models or occupy a scarce GPU.

Prior systems schedule or place agent calls [17, 26, 27, 39], reduce model-loading overhead [9], anticipate KV reuse [29, 49], or predict isolated executions [38]. Existing planners couple several decisions but exclude admission-boundary transformations from joint lifecycle, ordering, placement, and cumulative-memory planning. They therefore do not jointly evaluate a transformation’s cumulative effects on ready work, scarce devices, memory, and resident state.

The runtime must expose each resolved cross-workflow window, forecast readiness and device-specific costs, construct legal physical alternatives, and build a feasible graph-schedule plan that adapts to workflow events and telemetry.

Challenge 1: constructing legal physical executions. A logical agent graph fixes dependencies, roles, tool semantics, and externally visible effects, but leaves open how each activation is realized physically. The runtime must construct a bounded set of useful same-deployment fusions and model-lifecycle realizations while preserving logical dependencies, request isolation, and completed results.

Challenge 2: cross-device performance prediction. Physical-graph cost depends jointly on activation readiness, device-specific execution time, model-transition time, peak memory, and future deployment reuse. Exhaustive profiling does not scale across models, devices, sequence lengths, and serving configurations, while underestimation can reverse a device ranking or make an infeasible timeline appear feasible.

Challenge 3: feasible online graph-schedule construction. An alternative that is cheap in isolation can delay ready work, consume the only suitable GPU for a constrained agent, or violate capacity after model, request, engine, and transitional memory demands accumulate. The runtime must choose graph structure, placement, resource order, and state lifetime together, commit only a small executable prefix, and revise the uncommitted suffix when arrivals, tool returns, or telemetry invalidate the plan.

SAGEPILOT uses a resolved but not-yet-runnable workflow window to prepare physical execution without changing logical semantics. At each scheduling event, SAGERADAR forecasts activation-device costs and release times; SAGEGRAPH constructs legal fusion and model-lifecycle alternatives, including compatible actions shared across workflows; and SAGEWEAVER builds a feasible graph-schedule plan while protecting ready work. SAGEPILOT commits only through the next execution boundary and revises uncommitted physical decisions as workflow events and vLLM telemetry arrive, while preserving logical dependencies and results.

The main contributions of this work are¹:

- **Graph construction.** We construct bounded physical alternatives for concurrent multi-agent workflows through same-deployment chain fusion and compatible model-lifecycle actions, and expose them for joint planning with resource ordering and placement.
- **Cross-device performance prediction.** SAGERADAR predicts device-specific execution time, peak memory, and model-loading time and propagates execution estimates through workflow dependencies to forecast activation readiness.
- **Adaptive scheduling.** SAGEWEAVER constructs a feasible physical graph and schedule under live pool state and cumulative memory constraints, protects ready work, and reranks affected uncommitted decisions using runtime feedback.

2 Background and Motivation

2.1 Agent Workflows and Serving State

Multi-Agent workflows interleave model calls with inter-agent messages, tool results, and control-flow decisions, causing their execution graphs to emerge incrementally at runtime [27, 40]. We call each execution of a model node an *activation*. Each activation combines a role prompt, request input, and predecessor context; a repair round is therefore a new activation even when it reuses the same role and model.

As the graph unfolds, downstream activations often become visible before they can run. An activation is *ready* after all required predecessors complete and *near-ready* when its branch, role, model, immutable input prefix, and eligible devices are known, every predecessor is complete or running, and at least one remains running. A fan-in synthesizer

¹Code and data will be released upon acceptance.

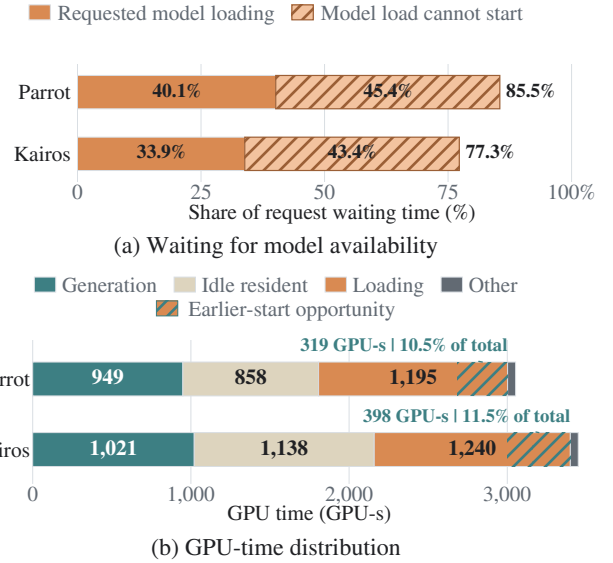


Figure 1. Model-lifecycle inefficiency under request-only scheduling. (a) Waiting for model availability. (b) GPU-time distribution; hatching marks loading that can start before request readiness.

can be near-ready while its inputs are produced, whereas a repair activation remains unknown until a test selects its branch. Ready activations and immediate near-ready successors form a bounded window that exposes useful future structure without assuming unresolved branches.

Private multi-Agent deployments keep sensitive inputs within organizational infrastructure [5, 20], where cost constraints limit spare capacity and elastic expansion. Concurrent workflows therefore share a small, fixed GPU pool whose devices may differ in compute throughput, memory capacity, and resident state. In this setting, early visibility matters because the same activation can incur different physical costs across devices. The logical graph determines when a result may be consumed, while resident weights, KV cache, prefixes, engine state, and device capacity determine how the activation can be realized. Retaining a deployment for a visible near-ready successor can avoid reloading; without one, the replica merely occupies scarce memory. Workflow progress therefore links readiness to model residency and device placement, enabling planning beyond individual requests.

2.2 Limits of Request-Level Execution

Existing workflow schedulers expose dependencies or prioritize requests but still act on individual model calls once they become ready [3, 26, 35]. They can reorder ready work, but do not jointly manage the backend working set or preserve physical continuity across logical node boundaries. We isolate these two limitations on a common serving substrate.

Workflow scheduling remains decoupled from model lifecycle management. Workflow information can change which ready request runs next, but not when the model required by a downstream node becomes available. We evaluate Parrot and Kairos on a common serving substrate and measure the resulting model-availability delay and lifecycle overhead.

Figure 1(a) decomposes model-availability delay into time spent waiting for an ongoing load and time during which the required load cannot start; together they account for 77.3–85.5% of request waiting under both baselines. Changing request priority therefore does not remove the dominant source of waiting. Figure 1(b) shows that model loading consumes more GPU time than token generation, while idle residency occupies another 28.1–33.0% of total GPU time. The hatched segments identify loading that workflow dependencies expose before request readiness, accounting for 10.5–11.5% of total GPU time. This interval is an opportunity bound rather than a measured speedup. The limitation is therefore not a particular request priority, but the separation between workflow scheduling and model lifecycle management. Addressing it requires treating model availability as part of workflow execution rather than as a reaction to each ready request.

Logical node boundaries create repeated physical admission. Dependency-aware scheduling still treats each model call as an independent scheduling unit. When adjacent nodes require identical model and serving configurations, every successor releases, requeues, and reacquires the backend. We measure this boundary for Parrot and Kairos while holding placement, admission, batching, and model-lifecycle mechanisms constant. QMSum has two parallel three-node Qwen3-4B lanes followed by three Qwen3-8B aggregation nodes; six of nine calls have a same-model successor [50]. A concurrent four-model MBPP repair workflow creates residency pressure on one A100-40GB and two V100-32GB GPUs [1]. Three repetitions yield 30 QMSum sessions per baseline.

Figure 2 shows a small median component—2.7–3.8 s, or 2–3%—but a large tail component: 97.1–155.1 s, or 39–42%, in the p95 sessions. Trace-confirmed loading accounts for 65.9 s and 130.7 s of the two p95 segments. Both baselines repeat admission at the same logical boundaries, identifying the physical boundary as the source of the delay. Fusion changes the admission unit: the chain competes at its head and retains separate generations under one backend admission. The fixed-trace components measure this opportunity; Section 4.5 reports the resulting net speedup under changed residency and interference.

2.3 Opportunity for Physical-Graph Execution

The experiments expose two axes absent from request scheduling: state lifetime and the physical scheduling unit. The

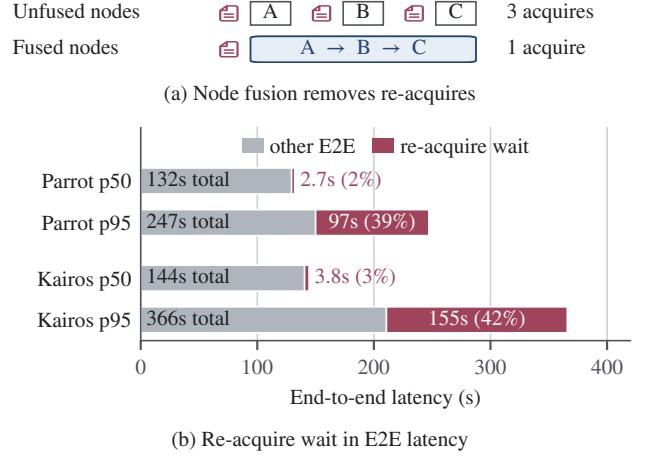


Figure 2. Repeated admission at same-model node boundaries. Highlighted segments are fixed-trace contributions of successor acquire waits, not measured fusion speedups.

realized logical window fixes dependencies and visible results, but not scheduling-unit boundaries, model-lifecycle actions, resource order, or placement. These choices form a mutable physical execution graph.

Near-ready preparation. Near-readiness makes model weights, serving configuration, and an immutable prefix determinate before the remaining predecessors complete. This interval exposes a preparation stage; SAGEWEAVER admits it only from residual capacity. Its generation and any external side effect still wait for the original logical dependencies. Consecutive nodes with identical model and serving configurations form one admitted unit while retaining separate prompts, generations, results, and commit points. Across workflows, requests with matching model and serving configurations and authorized isolation share one inference backend and its model residency; private KV state, completion events, and semantic graphs remain separate. Preparation, fusion, and cross-workflow sharing preserve workflow dependencies while changing physical realization.

Evaluate choices against cumulative heterogeneous state. A transformation that appears cheap in isolation becomes infeasible when its cumulative profile exceeds pool capacity. Loading exceeds memory when old weights, retained KV, engine buffers, and incoming state coexist, even if the final set fits. Placing a flexible activation on the only high-capacity GPU blocks a constrained successor; resource ordering preserves that device and reduces replacement and loading work. vLLM further reacts to KV pressure through queueing, preemption, and recomputation [22, 37]. Physical alternatives therefore require device-specific, time-varying resource profiles that capture their cumulative effects.

The runtime must predict sparse model–device profiles, construct bounded legal alternatives across workflows, and jointly select graph structure, placement, resource order, and state lifetime under live pool state. It must also limit commitment: tool returns, branch outcomes, completions, and serving telemetry invalidate future costs while the logical workflow remains unchanged. SAGERADAR, SAGEGRAPH, and SAGEWEAVER address these three requirements as the prediction, construction, and adaptive-selection stages of SAGEPILOT.

3 Design

3.1 Overview

Each multi-Agent workflow specifies what must happen and which results become visible, but leaves open how its model calls should share execution boundaries, replicas, or GPUs. Figure 3 shows how SAGEPILOT jointly turns these logical workflows into an executable graph–schedule plan. SAGERADAR predicts run time, peak memory, and loading time for each model–request configuration on every GPU type. Guided by these estimates, SAGEGRAPH fuses compatible same-deployment chains and exposes reusable or shared lifecycle actions without changing workflow semantics. SAGEWEAVER incrementally constructs a feasible physical graph and schedule by ranking lifecycle, replica, placement, and resource-order decisions on a common timeline, then commits the next executable prefix. As calls complete and replica state changes, the same process revises the uncommitted suffix.

3.2 Logical–Physical Execution Model

At time t , the scheduler observes the resolved portion $L_t^w = \langle V_t^w, E_t^w \rangle$ of each active workflow $w \in \Omega_t$, where each activation is a distinct invocation. SAGEGRAPH converts their union $L_t = \biguplus_{w \in \Omega_t} L_t^w$ into the fusion-normalized graph G_t^0 . Let F_t and N_t denote its ready and near-ready units. A unit belongs to F_t when all predecessors have completed; it belongs to N_t when every predecessor has completed or is running and at least one remains running. Completion estimates for running predecessors come from the scheduler’s committed timeline. These sets define the planning window $\mathcal{W}_t = G_t^0[F_t \cup N_t]$. A branch enters only after its control result resolves, so the window contains work on determined logical paths.

The window identifies what may execute next, but not how deployments become available or where units run. This distinction separates the logical frontier exposed by workflow progress from the physical choices made by the runtime. Let $U_t^0 = V(\mathcal{W}_t)$. A physical realization is $G_t^p = \langle U_t^0 \cup H_t^p, E_t^{L,p} \rangle$. Here, H_t^p contains device-symbolic loading, prefetching, and reclamation actions, while $E_t^{L,p}$ preserves logical dependencies and places each action before its consumers. The same logical window may therefore admit several realizations: reuse a resident replica, load a new one, or reclaim an idle

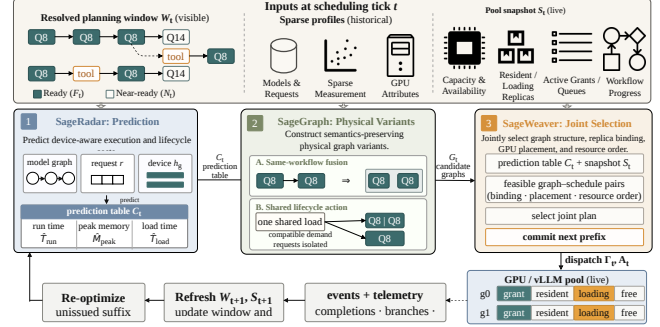


Figure 3. SAGEPILOT combines semantics-preserving same-workflow fusion, shared lifecycle actions across isolated workflows, device-aware predictions, and live pool state to construct and incrementally commit a feasible graph–schedule plan.

replica before loading. The graph prescribes the required lifecycle actions; its schedule π assigns devices, replicas, and reclamation victims, orders resource conflicts, and predicts start and completion times.

Snapshot S_t records GPU capacities, resident and loading replicas, active grants and queues, and workflow progress. A deployment identity d combines a resolved model and version with its serving configuration. Unit u requires identity $d(u)$ and model $m(u) = m(d(u))$; units with the same identity may reuse a replica while retaining per-activation request state and outputs.

An execution grant admits a unit to a selected replica, and its lease prevents that replica from being reclaimed until the unit completes. The next grant completion or model-lifecycle transition forms an execution boundary, where S_t is refreshed and the unissued suffix may be revised.

The invariants above define an admissible domain $\mathbb{G}_t$: every member is acyclic, preserves logical dependencies and visible values, respects deployment identity, and orders lifecycle actions before consumers. At each tick, SAGEWEAVER incrementally constructs a feasible (G_t, π_t) and dispatches its next executable prefix; arrivals, completions, predecessor updates, and model transitions refresh the window and rerank the uncommitted work.

3.3 SAGERADAR: Device-Aware Prediction

Rather than predicting workflow-level latency, SAGERADAR maps model structure $\mathcal{G}_m$, request configuration r , and device attributes $\mathbf{h}_g$ to

$$\left(\widehat{T}_{m,r,g}^{\text{run}}, \widehat{M}_{m,r,g}^{\text{peak}}, \widehat{T}_{d,g}^{\text{load}} \right) = \text{SageRadar}(\mathcal{G}_m, r, \mathbf{h}_g). \quad (1)$$

Here, d denotes the deployment identity for model m . The mapping yields consistent execution-cost estimates for each activation across model configurations and eligible devices, while separating request-dependent execution costs from

deployment-specific model-loading overhead. Run time supports placement and completion-time comparison, peak memory supports admission, and loading time prices loading, prefetching, and reclamation.

Graph–device prediction. SAGERADAR represents each model as an operator data-flow graph. Nodes describe operator type, arithmetic work, parameter size, and tensor shapes, while edges capture tensor dependencies and transferred data. Request features characterize the serving phase, batch size, and sequence lengths; device features characterize compute capability, memory capacity, and bandwidth. A graph encoder exchanges information along tensor edges and combines the pooled graph representation with the request and device context. Target-specific heads produce the three estimates in Equation 1, capturing request-dependent execution shapes and performance differences across heterogeneous GPUs.

Cached profiles and runtime feedback. Offline inference materializes run-time and memory profiles C_κ indexed by $\kappa = (d, g, b, \ell^{\text{in}}, \ell^{\text{out}})$, together with loading profiles $C_{d,g}^{\text{load}}$. At runtime, the input lookup selects the smallest bucket covering the observed prompt. Timing uses the smallest output bucket covering the empirical 90th-percentile output length of recent completions of the same logical workflow node, falling back to its configured limit when no history exists. Admission always uses the bucket covering the configured output limit. Both paths therefore use cached lookups rather than executing the graph encoder on the scheduling path.

The memory reserved for admission is $M_\kappa^{\text{adm}}(t) = \widehat{M}_\kappa^{\text{peak}} + \epsilon_M + \rho_\kappa(t)$, where ϵ_M covers unmodeled variation and $\rho_\kappa(t)$ increases after an observed out-of-memory failure. Runtime history can therefore refine completion estimates without reducing the memory reserved for feasibility.

Forecasting. Execution estimates become workflow-level timing information by propagating them through logical dependencies. The predicted release of unit u is

$$\widehat{r}_u = \begin{cases} t, & u \in F_t, \\ \max_{v \in \text{pred}_{G_t^0}(u)} \widehat{c}_v, & u \in N_t. \end{cases} \quad (2)$$

SAGEGRAPH sums the predicted run times of activations fused into one scheduling unit. SAGEWEAVER then combines release time, loading, execution, replica queues, and resource precedence into candidate-specific completion times. Observed completions update downstream releases and ordering decisions, while live pool state and cumulative-memory checks determine final feasibility.

3.4 SAGEGRAPH: Physical-Graph Construction

Given the planning window and SAGERADAR’s device estimates, SAGEGRAPH constructs physical candidates in two

stages. It first contracts logical nodes that can share a back-end grant within each workflow; after fusion, it derives the lifecycle actions required by the resulting units and coalesces compatible actions across workflows. For $L_t = (V_t, E_t)$, an edge (u, v) belongs to a maximal fusible chain when

$$u, v \in V_A, \quad \text{succ}_{L_t}(u) = \{v\}, \quad \text{pred}_{L_t}(v) = \{u\}, \quad (3)$$

$$d(u) = d(v), \quad \theta(u) \sim_{\text{cfg}} \theta(v).$$

Here, V_A denotes Agent nodes, $d(\cdot)$ the deployment identity, and $\sim_{\text{cfg}}$ configuration compatibility up to output-length limits. The one-to-one condition keeps branches and joins at chain boundaries. Contracting each maximal eligible chain yields $G_t^0 = \text{Fuse}(L_t)$. For $c = (v_1, \dots, v_h)$, external dependencies attach to its endpoints, while $v_1 \prec \dots \prec v_h$ executes under one grant and replica lease. Each activation retains its request configuration and passes its output to the next, preserving the original data flow.

In QMSum, three consecutive aggregation nodes use the same Qwen3-8B deployment and become $c = (v_1, v_2, v_3)$. The fused unit requires one grant on one replica, which remains available across all three activations, while their prompts, generations, and logical outputs remain distinct.

Because an earlier activation’s output may become part of a later activation’s input context, admission covers the largest context exposed along the complete chain. For activation v_i , let P_i be a bound on its fixed tokenized input, let $\mathcal{A}_i \subseteq \{1, \dots, i-1\}$ index the earlier outputs it consumes, and let O_i be its output limit; its input envelope is $\bar{I}_i = P_i + \sum_{j \in \mathcal{A}_i} O_j$. SAGEGRAPH assigns

$$I_c^{\text{adm}} = \max_{1 \leq i \leq h} \bar{I}_i, \quad O_c^{\text{adm}} = \max_{1 \leq i \leq h} O_i, \quad (4)$$

$$\widehat{T}_{c,g}^{\text{run}} = \sum_{i=1}^h \widehat{T}_{m(c), r_i, g}^{\text{run}}.$$

Here, r_i is the request configuration of v_i . The bounds I_c^{adm} and O_c^{adm} govern lease admission, while the summed duration predicts release. A length- h chain uses one grant and removes $h-1$ intermediate scheduling boundaries.

After fusion, SAGEGRAPH defines how each unit’s deployment can become available. For $X \in \{F, N\}$, let $\alpha_F = \text{Load}$ for ready work and $\alpha_N = \text{Prefetch}$ for near-ready work. The lifecycle alternatives are

$$\mathcal{H}_t^X(u) = \{\langle \alpha_X(d(u)) \rangle, \langle \text{Reclaim}, \alpha_X(d(u)) \rangle\}. \quad (5)$$

Lifecycle actions remain unplaced until scheduling. Reclamation selects a lease-free victim; resident binding reuses a replica, while loading creates or replaces one.

Let $h_{u,\rho} = 1$ when unit u selects lifecycle sequence ρ , with $\sum_\rho h_{u,\rho} \leq 1$. A unit without a sequence reuses a resident replica or is deferred. Combining these per-unit choices yields a physical realization: compatible transitions may share one device-symbolic action node, while distinct nodes represent replication.

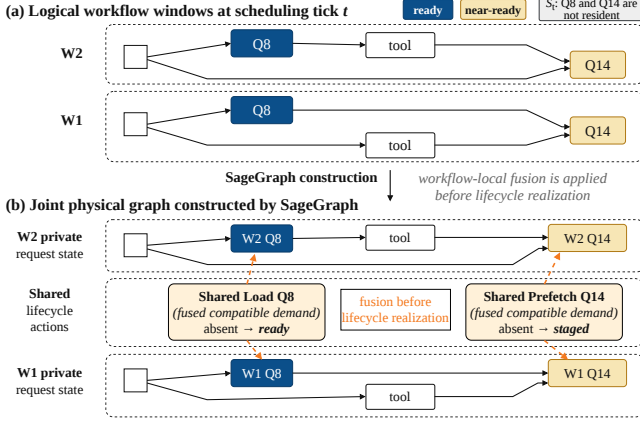


Figure 4. SAGEGRAPH constructs a joint physical graph at scheduling tick t . (a) W1 and W2 expose ready Q8 units and near-ready Q14 units while neither deployment is resident. (b) After workflow-local fusion normalization, compatible deployment demand across W1 and W2 shares one load that makes Q8 ready and one prefetch that stages Q14; request and tool state remain workflow-private.

Figure 4 illustrates this construction order and separation. At tick t , W1 and W2 each expose a ready Q8 unit and a near-ready Q14 unit, while S_t records both deployments as absent. SAGEGRAPH first fusion-normalizes each workflow while preserving its logical dependencies. It then coalesces only compatible lifecycle demand across workflows: one load makes Q8 available to both ready units, and one prefetch stages Q14 for both near-ready units. The execution nodes and their request and tool state remain workflow-private.

Under snapshot S_t , let $\mathcal{X}_t^H$ collect the lifecycle selections and compatible action-sharing choices that satisfy the preceding constraint and the invariants in Section 3.2. They induce the physical candidate space

$$\mathbb{G}_t = \{\text{Realize}(\mathcal{W}_t, h) : h \in \mathcal{X}_t^H\}. \quad (6)$$

This implicit domain guides SAGEWEAVER in constructing one realization jointly with its schedule.

3.5 SAGEWEAVER: Online Joint Graph-Schedule Construction

SAGEWEAVER implements an event-driven scheduling heuristic. It chooses lifecycle actions, replica and device bindings, and conflict order together because the value of loading, reusing, or reclaiming a model depends on the resulting contention and completion time.

Consider a window in which two workflows expose ready units for deployment d_1 , while a near-ready unit requires d_2 . Reusing a resident d_1 replica avoids loading, but may serialize both ready units behind its queue. Loading another d_1 replica increases parallelism, yet occupies memory that could prefetch d_2 ; reclaiming d_2 may accelerate the ready

work but incur a reload when the near-ready unit becomes eligible. Their effects become comparable once predicted releases, device-specific runtimes, replica queues, and memory transitions share one timeline. SAGEWEAVER ranks the resulting graph-schedule consequences on this timeline.

For operation $s \in V(G)$, let $\text{pred}_G(s)$ and $\text{rpred}_\pi(s)$ denote its graph and resource-order predecessors, and let $a_\pi(s)$ be its device. Its start follows both precedence relations,

$$\widehat{s}_s^{G,\pi} = \max \left\{ \widehat{r}_s, \max_{v \in \text{pred}_G(s)} \widehat{c}_v^{G,\pi}, \max_{v \in \text{rpred}_\pi(s)} \widehat{c}_v^{G,\pi} \right\}, \quad (7)$$

$$\widehat{c}_s^{G,\pi} = \widehat{s}_s^{G,\pi} + \widehat{T}_{s,a_\pi(s)}.$$

Here, $\widehat{r}_s$ follows Equation 2 for units and is the current time for lifecycle actions. The duration uses SAGERADAR's run or load estimate, or the reclamation estimate recorded in S_t . This common timeline propagates model preparation and placement contention into the predicted completion of their consumers.

A candidate placement must first fit the deployment and request. Let $\mathcal{D}$ be the accelerators, with type $g(a)$ and memory capacity C_a^M ; I_u, O_u are unit u 's input and output bounds, and $\kappa(u, g, v)$ indexes its prediction at batch size v . Its feasible domain is

$$\mathcal{D}_t(u, v) = \left\{ a \in \mathcal{D} \mid \begin{array}{l} I_u + O_u \leq L_{d(u)}^{\max} \\ M_{\kappa(u,g(a),v)}^{\text{adm}}(t) \leq C_a^M \end{array} \right\}, \quad (8)$$

where $L_{d(u)}^{\max}$ is the deployment's context limit and fused units use Equation 4. This domain is only the first feasibility cut. A fused unit can satisfy its context limit yet remain infeasible on a smaller GPU when its predicted admission memory exceeds capacity. Conversely, a device may enter the domain even when the deployment is not resident; the lifecycle choice and cumulative check below determine whether loading it fits the full timeline. Among the remaining devices, SageWeaver compares completion times obtained from release, device availability, model preparation, and execution.

Each selected unit must either reuse a resident replica or follow a lifecycle action placed on the same GPU. For selected batch size v_s , write $\mathcal{D}_t(s) = \mathcal{D}_t(s, v_s)$; a shared action uses the intersection of its consumers' domains. Let binary y_s select operation s , $x_{s,a}$ bind it to device a , and $\beta_{u,q}$ bind unit u to an accepting replica $q \in Q_t(u)$ hosted by $a(q)$. Let $\lambda_u(G)$ indicate a loading or prefetch predecessor. These variables couple operation selection to device placement and replica supply. The graph-schedule consistency constraints are

$$\begin{aligned} \sum_{a \in \mathcal{D}_t(s)} x_{s,a} &= y_s, & x_{h,a} &= x_{u,a}, & (h, u) &\in E^L(G), \\ \sum_{q \in Q_t(u)} \beta_{u,q} + \lambda_u(G) &= y_u, & \beta_{u,q} &\leq x_{u,a(q)}. \end{aligned} \quad (9)$$

A reclamation action analogously binds one lease-free victim on its device.

Placement may introduce contention between logically independent operations. For each pair $(i, j, a) \in \mathcal{K}_t^\pi$ placed on the same nonconcurrent resource, the ranked action order induces one orientation:

$$z_{ij,a} + z_{ji,a} = 1, \quad z_{ij,a}, z_{ji,a} \in \{0, 1\},$$

$$z_{ij,a} = 1 \Rightarrow \hat{s}_j^{G,\pi} \geq \hat{c}_i^{G,\pi}, \quad z_{ji,a} = 1 \Rightarrow \hat{s}_i^{G,\pi} \geq \hat{c}_j^{G,\pi}. \quad (10)$$

The combined precedence relation remains acyclic.

Memory must be checked cumulatively because individually feasible operations can still overlap: a new replica may load while resident models and active requests remain live. At each predicted transition boundary $\tau \in \mathcal{T}_t^\pi$, let $R_{a,t}^{G,\pi}(\tau)$ be resident and transitional model memory, and $M_{s,a}^{\text{inc}}$ the incremental request memory of active unit s . Memory feasibility requires

$$R_{a,t}^{G,\pi}(\tau) + \sum_s x_{s,a} M_{s,a}^{\text{inc}} \mathbf{1}[\hat{s}_s^{G,\pi} \leq \tau < \hat{c}_s^{G,\pi}] \leq C_a^M, \quad \forall a, \tau, \quad (11)$$

and concurrent units assigned to replica q additionally respect its batch limit B_q . This transition-by-transition check rejects overlaps that exceed device capacity, turning a physical realization into an executable schedule that couples lifecycle choice, placement, resource order, and request execution on one predicted timeline.

Within this feasible region, SAGEWEAVER first exploits capacity that can serve ready work without another model load. A resident accepting replica provides immediate reuse; when loading is necessary, its value grows with the ready work it unlocks relative to predicted loading time, after accounting for priority and aging. Scale-out is considered only when a new replica completes the unit earlier than the current queues, subject to the replica cap. Pool-wide demand and queue state allow one residency decision to serve several workflows.

Reclamation considers idle, lease-free replicas, preferring redundant copies and then distant next use relative to reload cost; least-recently-used order handles missing estimates. A near-ready deployment becomes eligible for prefetch when its remaining release time approaches loading time plus dispatch slack, but only if prefetching does not delay ready work in the selected plan.

Planning remains tick-local: $\mathcal{W}_t$ contains only ready units and immediate near-ready successors. SAGEWEAVER processes ready units by priority and aging, first reusing accepting replicas and then choosing one feasible binding per remaining unit. Required loads, scale-out, and lease-safe reclamation are ranked by predicted benefit; near-ready prefetch is considered only when no ready action can advance. Resource conflicts inherit this action order, and each accepted decision is checked against Equations 9–11.

SAGEWEAVER ranks decisions first by ready-work coverage across priority classes, then by completion time, and finally by timely prefetching and lifecycle overhead. The

Algorithm 1: Online Joint Graph–Schedule Construction

Input: Planning window $\mathcal{W}_t$; ready/near-ready sets F_t, N_t ; snapshot S_t ; prediction table C

Output: Constructed plan (G_t, π_t) ; grants Γ_t ; lifecycle actions $\mathcal{A}_t$

```

1:  $R \leftarrow \text{RankReady}(F_t, S_t)$  ▷ Priority and aging.
2:  $\Gamma_t, \mathcal{L}, \mathcal{B} \leftarrow \emptyset, \emptyset, \emptyset$ 
3: for  $u \in R$  do
4:    $b_u \leftarrow \text{BestFeasibleBinding}(u, S_t, C)$  ▷ One binding.
5:   if  $b_u$  reuses an accepting resident replica then
6:      $\Gamma_t \leftarrow \Gamma_t \cup \{(u, b_u)\}$ 
7:   else if  $b_u \neq \perp$  then
8:      $\mathcal{L} \leftarrow \mathcal{L} \cup \{(u, b_u)\}$ 
9:   else
10:     $\mathcal{B} \leftarrow \mathcal{B} \cup \{u\}$ 
11:  $\mathcal{A}_t \leftarrow \text{PlanReady}(\mathcal{L}, \mathcal{B}, S_t, C)$  ▷ Load, scale-out, or reclaim.
12: if  $\Gamma_t = \emptyset \wedge \mathcal{A}_t = \emptyset$  then
13:    $\mathcal{A}_t \leftarrow \text{PlanPrefetch}(N_t, S_t, C)$ 
14:  $(G_t, \pi_t) \leftarrow \text{Propagate}(\mathcal{W}_t, \Gamma_t, \mathcal{A}_t, S_t)$  ▷ Order conflicts; check feasibility.
15:  $(\Gamma_t, \mathcal{A}_t) \leftarrow \text{Prefix}_t(G_t, \pi_t, S_t)$ 
16: return  $(G_t, \pi_t, \Gamma_t, \mathcal{A}_t)$ 

```

map $k_t : F_t \rightarrow \{1, \dots, K\}$ assigns each ready unit a priority and aging level. For the constructed plan, A_k counts admitted level- k units, $\hat{C}_F$ is their latest completion, P_N counts timely prefetches, and H_{life} combines ready-work loading delay with reclamation-induced reload cost. The lexicographic ranking key is

$$\Phi_t(G, \pi) = (-A_1, \dots, -A_K, \hat{C}_F, -P_N, H_{\text{life}}). \quad (12)$$

Algorithm 1 applies this ordering while constructing a feasible plan and committing its next executable prefix.

Algorithm 1 scans ready units in ranked order. For each unit, BestFeasibleBinding filters eligible devices and returns at most one binding. Resident replicas are reused first; remaining load, scale-out, and lease-safe reclamation actions are ranked by predicted benefit. Prefetch is considered only when no ready action can advance. Propagate serializes co-located operations and checks Equations 9–11 before Prefix_t commits the next executable actions. Subsequent events refresh $\mathcal{W}_t$ and S_t and rerank uncommitted work.

4 Evaluation

4.1 Experimental Setup

Testbed. The testbed comprises three heterogeneous nodes. Each of two servers has two Intel Xeon Gold 6148 processors, with 20 cores per socket, and four NVIDIA V100 GPUs with 32 GB of memory. The remaining server has two Intel Xeon Silver 4216 processors, with 16 cores per socket, and four NVIDIA A100 GPUs with 40 GB of memory. All servers run CUDA 12 with the same software environment: vLLM 0.10.2, PyTorch 2.8.0, and Transformers 4.55.2.

Models and Runtime. Five Qwen3 models (0.6B, 1.7B, 4B, 8B, and 14B) share one vLLM serving configuration [41].

Each resident replica occupies one GPU and serves compatible requests through continuous batching; replicas are loaded and reclaimed on demand. All systems use the same inference engine, data type, and serving configuration, so their differences arise from physical-graph construction, lifecycle management, and scheduling. Each arrival trace is generated once with seed 42 and replayed identically across all systems. Each system–trace configuration is executed five times, and we report the mean of the five run-level measurements.

Workloads and Datasets. We construct three application-level workloads by pairing a public benchmark with a representative Agent DAG. The datasets provide realistic task contents and input-length distributions, while the DAGs determine inter-call dependencies, parallelism, tool use, and model reuse. GSM8K [6] contains diverse grade-school math problems requiring multi-step reasoning; its ensemble workflow runs five heterogeneous solvers in parallel, aggregates their answers by majority vote, and refines the result. Sanitized MBPP [1] provides Python programming tasks with executable tests; its tool-augmented repair workflow performs code generation, testing, diagnosis, review, repair, and final validation while passing execution feedback across stages. QMSum [50] contains query-focused summaries of long, multi-domain meeting transcripts; its hierarchical workflow uses two parallel three-stage rolling-summary chains followed by a three-stage aggregation chain. Together, these workloads cover fork–join parallelism, tool-interleaved dependencies, long-context processing, fan-in aggregation, and repeated-model chains.

Each comparison uses identical workflow inputs, model assignments, arrival traces, and inference configurations.

Baselines. The evaluation compares default SAGEPILOT with two scheduling baselines and two predictor substitutions.

- **Parrot** [26] introduces Semantic Variables that annotate request inputs and outputs, connect multiple LLM calls into a dataflow, and expose cross-request correlations to the serving system.
- **Kairos** [3] combines a workflow-aware priority scheduler with a memory-aware dispatcher to reduce end-to-end latency when agents share loaded LLM instances.

Formula predictor. This variant replaces SAGERADAR with a static size-and-bandwidth estimator in the style of LLM-Fit [18]; it derives runtime and memory estimates from model weights, KV-cache demand, and device bandwidth.

GBDT predictor. This variant replaces SAGERADAR with LightGBM [19], trained on scalar model, request, and device features including model size, layer count, batch size, sequence lengths, and memory bandwidth.

All five configurations use the same vLLM execution, placement, batching, and model-loading substrate. The two predictor variants retain SAGEGRAPH and SAGEWEAVER unchanged and therefore isolate prediction quality rather than define standalone schedulers.

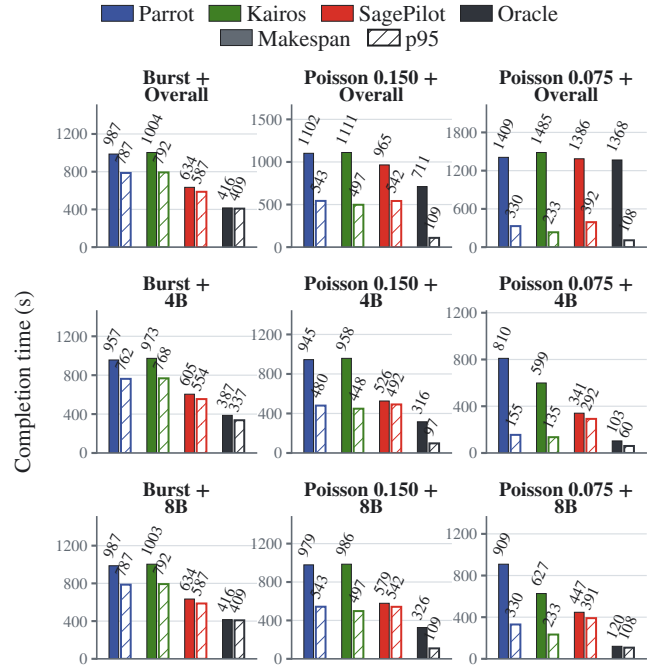


Figure 5. End-to-end completion time under burst and Poisson arrivals.

4.2 End-to-End Performance

The workload contains 180 sessions, 60 per scenario. Poisson arrivals use per-scenario rates of 0.150 and 0.075 sessions/s; the burst setting submits all sessions together. Overall makespan measures the interval from the first arrival until the last session finishes; overall p95 is the 95th percentile of per-session completion latency. Model-scoped completion metrics include DAG dependencies, queuing, model loading, and execution rather than only inference time.

The Oracle is a trace-conditioned offline scheduler with complete knowledge of task and model-loading durations. It optimizes the displayed completion metrics subject to the same arrival trace, DAG dependencies, GPU capacity, model compatibility, and serving constraints as the measured systems. We use it as a strong offline reference rather than a realizable online policy or a strict lower bound.

SAGEPILOT finishes the workload in 634.2 s, 35.8% and 36.8% earlier than Parrot and Kairos, respectively. Its overall p95 is 587.0 s, 25.4–25.9% below the two baselines, and its 4B and 8B paths also finish first.

SAGEPILOT also achieves the lowest overall makespan at both Poisson rates, finishing in 965.1 and 1386.4 s at 0.150 and 0.075 sessions/s, respectively. These results correspond to reductions of 12.4–13.1% and 1.6–6.6% over Parrot and Kairos. Some Poisson p95 and model-scoped results favor another policy: SAGEPILOT optimizes global completion and can trade a local workflow tail for earlier workload drain.

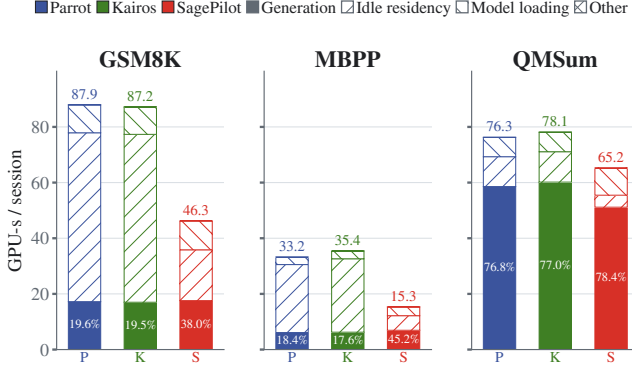


Figure 6. GPU time per completed session by workflow scenario under burst arrivals.

Thus, SAGEPILOT drains the workload earlier in every arrival setting, although its global objective does not always minimize per-session or model-scoped tails under dispersed arrivals.

4.3 GPU-Time Efficiency

We next use the same 180-session burst workload as Section 4.2 to report cluster-wide GPU time per completed session for each workflow scenario. We attribute shared lifecycle time to scenarios by their share of each model’s generation time. We partition GPU time into generation, idle residency, model loading, and other activity. Generation is time spent generating tokens, whereas idle residency is time during which loaded model weights occupy GPU memory without generating tokens. Model loading is time spent preparing models, and other activity is the remaining GPU time. Unlike a percentage-only breakdown, GPU time per completed session captures both how long GPU capacity is occupied and how that capacity is used.

Figure 6 shows that SAGEPILOT uses less GPU time for all three workflow scenarios. For GSM8K and MBPP, it uses 46.26 and 15.33 GPU-s per completed session, 46.9–47.4% and 53.8–56.7% less than the two baselines, respectively. For QMSum, it uses 65.25 GPU-s, a reduction of 14.5–16.5%. Averaged across all 180 sessions, SAGEPILOT saves up to 24.63 GPU-s per completed session over the two baselines.

The reduction comes primarily from idle residency. SAGEPILOT lowers it from 60.32–60.68 to 18.27 GPU-s for GSM8K, from 24.43–26.39 to 5.25 for MBPP, and from 10.74–10.92 to 4.31 for QMSum.

4.4 SageRadar Prediction Accuracy

SAGEWEAVER consumes cached resource profiles to compare the cost of running an Agent task on different devices and determine whether that task fits an available execution window. Because a numerical error need not change either choice, the evaluation measures both numerical fidelity and downstream

Table 1. Prediction fidelity and decision accuracy of the three predictors across different prediction targets.

Method	WAPE (%) ↓			Decision accuracy (%) ↑	
	Loading	Runtime	Peak VRAM	Ordering	Window fit
SAGERADAR	8.05	7.25	14.08	97.94	97.82
Formula	27.21	15.29	11.98	96.37	96.82
GBDT	24.35	11.05	15.70	96.53	97.18

decision fidelity at the predictor–scheduler interface [31]. 2,798 measured Qwen3 model deployment configurations were used to form the test set.

Weighted absolute percentage error (WAPE) measures the numerical fidelity of loading-time, runtime, and peak-VRAM predictions by summing absolute errors over the test set and dividing by the sum of their measured values. For downstream decision fidelity, *ordering* measures whether predicted and measured runtimes agree on which of two configurations is faster [10]. *Window fit* measures whether the prediction and measurement agree that a configuration can finish within a given window. Both metrics compare choices induced by cached predictions with those induced by measured profiles. The window is the median measured runtime of the held-out configurations, 48.85 s. Together, they provide a closer proxy than point-estimate error alone for how cached predictions affect SAGEWEAVER’s decisions, while Section 4.2 measures the realized system-level impact.

Table 1 shows that SAGERADAR achieves the lowest loading-time and runtime WAPE, at 8.05% and 7.25%, respectively. This reduces loading-time WAPE by 70.4% over the formula-based predictor and 66.9% over GBDT; the corresponding runtime reductions are 52.6% and 34.4%. Although all three predictors exceed 96% decision accuracy, SAGERADAR achieves the highest fidelity: it reproduces 97.94% of measured runtime orderings and 97.82% of measured window-fit outcomes, improvements of 1.41–1.57 and 0.64–1.00 percentage points over the baselines, respectively. The formula-based predictor has the lowest peak-VRAM WAPE; SAGERADAR is 2.10 percentage points higher but combines competitive VRAM fidelity with the best runtime and decision accuracy.

4.5 Predictor and Component Ablations

The analysis uses the same 180-session workload and three arrival processes as Figure 5 to compare full SAGEPILOT against two predictor substitutions and three component ablations. The predictor substitutions use the formula-based or GBDT predictor while leaving the rest of SAGEPILOT unchanged. The component ablations disable SAGEGRAPH node fusion, disable near-ready prefetching, or make lifecycle decisions per workflow. The per-workflow lifecycle condition continues to share loaded replicas across workflows, but each workflow separately estimates reuse, orders waiting requests, and evaluates loading benefit. All three component

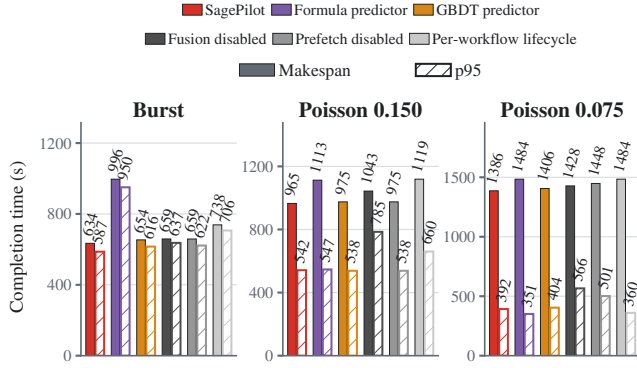


Figure 7. End-to-end performance of predictor substitutions and component ablations.

ablations retain SAGERADAR and the remaining SAGEPILOT mechanisms.

Across the three arrival processes, full SAGEPILOT reduces makespan by 1.0–3.0% over the GBDT substitution and 6.6–36.3% over the formula-based substitution (Figure 7). Among the component ablations, restricting lifecycle decisions to one workflow increases makespan by 7.1–16.4%, while disabling node fusion raises p95 by 8.5–44.9% and makespan by 3.0–8.1%. Disabling prefetching also increases makespan in every setting. These results show that cross-workflow lifecycle coordination provides the largest makespan benefit, node fusion provides the strongest tail-latency benefit, and prefetching contributes consistently across arrival processes.

5 Related Work

Prediction. DNNPerf [12] uses a GNN to predict training time and GPU memory; PerfSeer [48] adds node, edge, and global features for multiple metrics. nn-Meter [47] sums kernel estimates on edge devices; Habitat [44] transfers measurements across GPUs; NeuSight [25] predicts unseen model-GPU pairs from hardware-bounded tiles. HELP [24] adapts from a few target measurements; LitePred [8] transfers kernel predictors across similar platforms. SAGERADAR predicts run time, peak memory, and loading per model-request-device candidate for execution and lifecycle decisions.

DAG execution and transformation. DryadLINQ [45] compiles LINQ into distributed graphs; Tez [32] supplies a DAG runtime. Faastlane [21] co-locates serverless functions, while WiseFuse [28] profiles DAG fusion and task bundling. GMorph [42] searches feature-sharing mutations across pretrained DNNs; Mercury [13] compiles remote-memory-aware multi-GPU operator schedules. SAGEGRAPH preserves Agent semantics while fusing deployment-identical chains and adding lifecycle vertices; SAGEWEAVER jointly selects bindings, placement, resource order, and cumulative memory.

Serving-state management. vLLM [22] uses PagedAttention for shareable KV caches; ServerlessLLM [9] combines optimized checkpoints, tiered storage, and locality-aware scheduling to reduce startup latency. ELORA [34] jointly manages dependent LoRA-KV state; CrossPool [43] disaggregates stable FFN weights and transient KV state. KVFlow [29] evicts and prefetches KV state; PBKV [49] predicts reuse; TokenCake [2] protects critical-agent KV state and offloads it during tool calls. Concurrent workflows additionally require state transitions to account for dependency-driven readiness, future model demand, and heterogeneous device availability across the pool. SAGEPILOT schedules model lifecycles against workflow reuse, contention, and cumulative pool occupancy.

Agent serving and scheduling. Parrot [26] uses Semantic Variables to connect LLM-call producers and consumers, exposing inter-request dependencies to the scheduler. Autellix [27] prioritizes calls by agent-program progress; Kairos [3] combines workflow-aware priority with memory-aware dispatch. Ayo [35] exposes primitive-level parallelism; Aragog [7] selects accuracy-equivalent routes under load. Halo [33] consolidates batched DAGs for computation, cache reuse, and placement; SAGA [14] schedules workflows using predicted cross-tool-call reuse; Maestro [39] coordinates model co-location, routing, and priority across clusters. Most schedule ready calls or DAG units, leaving physical graph construction and future lifecycle demand outside a unified planning loop. SAGEWEAVER jointly chooses fusion, lifecycle actions, binding, placement, and resource order under cumulative pool state and replans from feedback.

6 Conclusion

We present SAGEPILOT, a prediction-guided runtime for concurrent Agent workflows on heterogeneous GPUs. SAGEPILOT separates logical semantics from state-dependent physical execution. SAGERADAR predicts activation-device costs and readiness across model configurations and heterogeneous devices; SAGEGRAPH constructs legal fusion and model-lifecycle alternatives; and SAGEWEAVER selects and schedules these alternatives under live pool state, revising uncommitted decisions from feedback. Across a workload spanning three workflow scenarios, SAGEPILOT reduces burst makespan and overall p95 completion latency by up to 36.8% and 25.9%, respectively, over Parrot and Kairos, while saving up to 24.63 GPU-s per completed session. These results show the value of joint physical-graph and lifecycle orchestration.

Acknowledgments

We use generative AI tools, including Writefull and ChatGPT, to support the proofreading and language refinement. These tools help us identify grammatical errors, improve clarity and readability.

References

- [1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021). doi:10.48550/arXiv.2108.07732
- [2] Zhuohang Bian, Feiyang Wu, Zhuoran Li, Teng Ma, and Youwei Zhuo. 2025. TokenCake: A KV-Cache-Centric Serving Framework for LLM-Based Multi-Agent Applications. *arXiv preprint arXiv:2510.18586*. doi:10.48550/arXiv.2510.18586
- [3] Jinyuan Chen, Jiuchen Shi, Quan Chen, and Minyi Guo. 2025. Kairos: Low-latency Multi-Agent Serving with Shared LLMs and Excessive Loads in the Public Cloud. *arXiv preprint arXiv:2508.06948* (2025). doi:10.48550/arXiv.2508.06948
- [4] Zehui Chen, Kuikun Liu, Qiuchen Wang, Jiangning Liu, Wenwei Zhang, Kai Chen, and Feng Zhao. 2025. MindSearch: Mimicking Human Minds Elicits Deep AI Searcher. In *International Conference on Learning Representations (ICLR)*, Vol. 2025. 90007–90029. https://proceedings.iclr.cc/paper_files/paper/2025/file/e01c431bbb83153632c0dcfaf8ccda0a-Paper-Conference.pdf
- [5] Marcin Chrapek, Marcin Copik, Etienne Mettaz, and Torsten Hoefler. 2025. Confidential LLM Inference: Performance and Cost Across CPU and GPU TEEs. In *2025 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 84–98. doi:10.1109/IISWC66894.2025.00017
- [6] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168* (2021). doi:10.48550/arXiv.2110.14168
- [7] Yinwei Dai, Zhuofu Chen, Anand Iyer, and Ravi Netravali. 2025. Aragog: Just-in-Time Model Routing for Scalable Serving of Agentic Workflows. *arXiv preprint arXiv:2511.20975*. doi:10.48550/arXiv.2511.20975
- [8] Chengquan Feng, Li Lyna Zhang, Yuanchi Liu, Jiahang Xu, Chengruidong Zhang, Zhiyuan Wang, Ting Cao, Mao Yang, and Haisheng Tan. 2024. LitePred: Transferable and Scalable Latency Prediction for Hardware-Aware Neural Architecture Search. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, 1463–1477. <https://www.usenix.org/conference/nsdi24/presentation/feng-chengquan>
- [9] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 135–153. <https://www.usenix.org/conference/osdi24/presentation/fu>
- [10] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. 2024. Efficient LLM Scheduling by Learning to Rank. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 59006–59029. doi:10.52202/079017-1882
- [11] Dawei Gao, Zitao Li, Xuchen Pan, Weirui Kuang, Zhijian Ma, Bingchen Qian, Fei Wei, Wenhao Zhang, Yuexiang Xie, Daoyuan Chen, Liuyi Yao, Hongyi Peng, Zeyu Zhang, Lin Zhu, Chen Cheng, Hongzhu Shi, Yaliang Li, Bolin Ding, and Jingren Zhou. 2024. AgentScope: A Flexible yet Robust Multi-Agent Platform. *arXiv preprint arXiv:2402.14034*. doi:10.48550/arXiv.2402.14034
- [12] Yanjie Gao, Xianyu Gu, Hongyu Zhang, Haoxiang Lin, and Mao Yang. 2023. Runtime Performance Prediction for Deep Learning Models with Graph Neural Network. In *Proceedings of the 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 368–380. doi:10.1109/ICSE-SEIP58684.2023.00039
- [13] Yue Guan, Xinwei Qiang, Zaifeng Pan, Daniels Johnson, Yuanwei Fang, Keren Zhou, Yuke Wang, Wanlu Li, Yufei Ding, and Adnan Aziz. 2025. Mercury: Unlocking Multi-GPU Operator Optimization for LLMs via Remote Memory Scheduling. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25)*. ACM, 1046–1061. doi:10.1145/3731569.3764798
- [14] Dongxin Guo, Jikun Wu, and Siu Ming Yiu. 2026. SAGA: Workflow-Atomic Scheduling for AI Agent Inference on GPU Clusters. *arXiv preprint arXiv:2605.00528*. doi:10.48550/arXiv.2605.00528
- [15] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qishi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature* 645, 8081 (2025), 633–638. doi:10.1038/s41586-025-09422-z
- [16] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xianwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *International Conference on Learning Representations (ICLR)*, Vol. 2024. 23247–23275. https://proceedings.iclr.cc/paper_files/paper/2024/file/6507b115562bb0a305f1958ccc87355a-Paper-Conference.pdf
- [17] Zirui Huang, Yi-Xiang Hu, Feng Wu, and Xiang-Yang Li. 2026. FATE: Future-State-Aware Scheduling for Heterogeneous LLM Workflows. *arXiv preprint arXiv:2605.07238*. doi:10.48550/arXiv.2605.07238
- [18] Alex Jones. 2026. llmfit. Official GitHub repository. Accessed: 2026-08-03. <https://github.com/AlexsJones/llmfit>
- [19] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly

- Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*, Vol. 30. Curran Associates, Inc., 3146–3154. https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html
- [20] Jonathan Knoop and Hendrik Holtmann. 2026. Private LLM Inference on Consumer Blackwell GPUs: A Practical Guide for Cost-Effective Local Deployment in SMEs. arXiv preprint arXiv:2601.09527. doi:10.48550/arXiv.2601.09527
- [21] Swaroop Kotni, Ajay Nayak, Vinod Ganapathy, and Arkaprava Basu. 2021. Faastlane: Accelerating Function-as-a-Service Workflows. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, 805–820. <https://www.usenix.org/conference/atc21/presentation/kotni>
- [22] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP)*. 611–626. doi:10.1145/3600006.3613165
- [23] LangChain. 2026. LangGraph Overview. LangChain Documentation. Last updated: 2026-07-24; accessed: 2026-08-03. <https://docs.langchain.com/oss/python/langgraph/overview>
- [24] Hayeon Lee, Sewoong Lee, Song Chong, and Sung Ju Hwang. 2021. Hardware-adaptive Efficient Latency Prediction for NAS via Meta-Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34. Curran Associates, Inc., 27016–27028. <https://proceedings.neurips.cc/paper/2021/hash/e3251075554389fe91d17a794861d47b-Abstract.html>
- [25] Seonho Lee, Amar Phanishayee, and Divya Mahajan. 2025. Forecasting GPU Performance for Deep Learning Training and Inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 493–508. doi:10.1145/3669940.3707265
- [26] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 929–945. <https://www.usenix.org/conference/osdi24/presentation/lin-chaofan>
- [27] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. 2025. Autellix: An Efficient Serving Engine for LLM Agents as General Programs. arXiv preprint arXiv:2502.13965. doi:10.48550/arXiv.2502.13965
- [28] Ashraf Mahgoub, Edgardo Barsallo Yi, Karthick Shankar, Eshaan Minocha, Sameh Elnikety, Saurabh Bagchi, and Somali Chaterji. 2022. WiseFuse: Workload Characterization and DAG Transformation for Serverless Workflows. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 6, 2 (2022), 1–28. doi:10.1145/3530892
- [29] Zaifeng Pan, Ajikumar Dahyalal Patel, Yipeng Shen, Zhengding Hu, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. 2025. KVFlow: Efficient Prefix Caching for Accelerating LLM-Based Multi-Agent Workflows. In *Advances in Neural Information Processing Systems*, Vol. 38. Curran Associates, Inc., 126246–126265. https://proceedings.neurips.cc/paper_files/paper/2025/file/b7971d31a7d5eb0f1eed2f8f6f368195-Paper-Conference.pdf
- [30] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 15174–15186. doi:10.18653/v1/2024.acl-long.810
- [31] Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Saurabh Jha, Chen Wang, Hubertus Franke, Zbigniew Kalbarczyk, Tamer Başar, and Ravishankar K. Iyer. 2024. Power-aware Deep Learning Model Serving with μ -Serve. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 75–93. <https://www.usenix.org/conference/atc24/presentation/qiu>
- [32] Bikas Saha, Hitesh Shah, Siddharth Seth, Gopal Vijayaraghavan, Arun C. Murthy, and Carlo Curino. 2015. Apache Tez: A Unifying Framework for Modeling and Building Data Processing Applications. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. ACM, 1357–1369. doi:10.1145/2723372.2742790
- [33] Junyi Shen, Noppanat Wadlom, and Yao Lu. 2025. Batch Query Processing and Optimization for Agentic Workflows. arXiv preprint arXiv:2509.02121. doi:10.48550/arXiv.2509.02121
- [34] Jiuchen Shi, Hang Zhang, Yixiao Wang, Quan Chen, Yizhou Shan, Kaihua Fu, Wei Wang, and Minyi Guo. 2026. ELORA: Efficient LoRA and KV Cache Management for Multi-LoRA LLM Serving. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1–14. doi:10.1109/HPCA68181.2026.11408492
- [35] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. 2025. Towards End-to-End Optimization of LLM-based Applications with Ayo. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 2*. 1302–1316. doi:10.1145/3676641.3716278
- [36] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv preprint arXiv:2302.13971 (2023). doi:10.48550/arXiv.2302.13971
- [37] vLLM Team. 2026. Optimization and Tuning. vLLM Documentation. Last updated: 2026-07-07; accessed: 2026-08-03. <https://docs.vllm.ai/en/latest/configuration/optimization/>
- [38] Haoxin Wang, Xiaolong Tu, Hongyu Ke, Huirong Chai, Dawei Chen, and Kyungtae Han. 2025. Im-Meter: Unveiling Runtime Inference Latency for On-Device Language Models. In *Proceedings of the Tenth ACM/IEEE Symposium on Edge Computing (SEC '25)*. ACM, 1–17. doi:10.1145/3769102.3770614
- [39] Jinghao Wang, Xiao Zhou, Xiaoyang Sun, Yihui Zhang, Yilong Li, Tianyu Wo, Xu Wang, Chunming Hu, and Renyu Yang. 2026. Maestro: Workload-Aware Cross-Cluster Scheduling for LLM-Based Multi-Agent Systems. arXiv preprint arXiv:2606.12950. Accepted to the 46th IEEE International Conference on Distributed Computing Systems (ICDCS 2026). doi:10.48550/arXiv.2606.12950
- [40] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryan W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv preprint arXiv:2308.08155. doi:10.48550/arXiv.2308.08155
- [41] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruizhe Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. arXiv preprint arXiv:2505.09388 (2025). doi:10.48550/arXiv.2505.09388
- [42] Qizheng Yang, Tianyi Yang, Mingcan Xiang, Lijun Zhang, Haoliang Wang, Marco Serafini, and Hui Guan. 2024. GMorph: Accelerating

- Multi-DNN Inference via Model Fusion. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys '24)*. ACM, 505–523. doi:10.1145/3627703.3650074
- [43] Zhuoren Ye, Tianyu Wo, Dinghao Xue, Mingming Zhang, Yuchen Teng, Chunming Hu, and Renyu Yang. 2026. CrossPool: Efficient Multi-LLM Serving for Cold MoE Models through KV-Cache and Weight Disaggregation. arXiv preprint arXiv:2606.24506. doi:10.48550/arXiv.2606.24506
- [44] Geoffrey X. Yu, Yubo Gao, Pavel Golikov, and Gennady Pekhimenko. 2021. Habitat: A Runtime-Based Computational Performance Predictor for Deep Neural Network Training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, 503–521. <https://www.usenix.org/conference/atc21/presentation/yu>
- [45] Yuan Yu, Michael Isard, Dennis Fetterly, Mihai Budiu, Úlfar Erlingsson, Pradeep Kumar Gunda, and Jon Currey. 2008. DryadLINQ: A System for General-Purpose Distributed Data-Parallel Computing Using a High-Level Language. In *8th USENIX Symposium on Operating Systems Design and Implementation (OSDI 08)*. USENIX Association, 1–14. <https://www.usenix.org/conference/osdi-08/dryadlinq-system-general-purpose-distributed-data-parallel-computing-using-high>
- [46] Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared Quincy Davis, Heather Miller, Chris Potts, James Zou, Michael Carbin, Jonathan Frankle, Naveen Rao, and Ali Ghodsi. 2024. The Shift from Models to Compound AI Systems. Berkeley Artificial Intelligence Research (BAIR) Blog. <https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/>
- [47] Li Lyna Zhang, Shihao Han, Jianyu Wei, Ningxin Zheng, Ting Cao, Yuqing Yang, and Yunxin Liu. 2021. nn-Meter: Towards Accurate Latency Prediction of Deep-Learning Model Inference on Diverse Edge Devices. In *Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys)*. 81–93. doi:10.1145/3458864.3467882
- [48] Xinlong Zhao, Jiande Sun, Jia Zhang, Tong Liu, and Ke Liu. 2025. PerfSeer: An Efficient and Accurate Deep Learning Models Performance Predictor. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI-25)*. International Joint Conferences on Artificial Intelligence Organization, 7128–7136. doi:10.24963/ijcai.2025/793
- [49] Haoyu Zheng, Fangcheng Fu, Jia Wu, Binhang Yuan, Yongqiang Zhang, Hao Wang, Yuanyuan Zhu, Xiao Yan, and Jiawei Jiang. 2026. Efficient Serving for Dynamic Agent Workflows with Prediction-Based KV-Cache Management. arXiv preprint arXiv:2605.06472. doi:10.48550/arXiv.2605.06472
- [50] Ming Zhong, Da Yin, Tao Yu, Ahmad Zaidi, Mutethia Mutuma, Rahul Jha, Ahmed Hassan Awadallah, Asli Celikyilmaz, Yang Liu, Xipeng Qiu, and Dragomir Radev. 2021. QMSum: A New Benchmark for Query-based Multi-domain Meeting Summarization. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 5905–5921. doi:10.18653/v1/2021.naacl-main.472